



UNGDOMMENS NATURVIDENSKABELIGE FORENING

Specialiserede Modeller

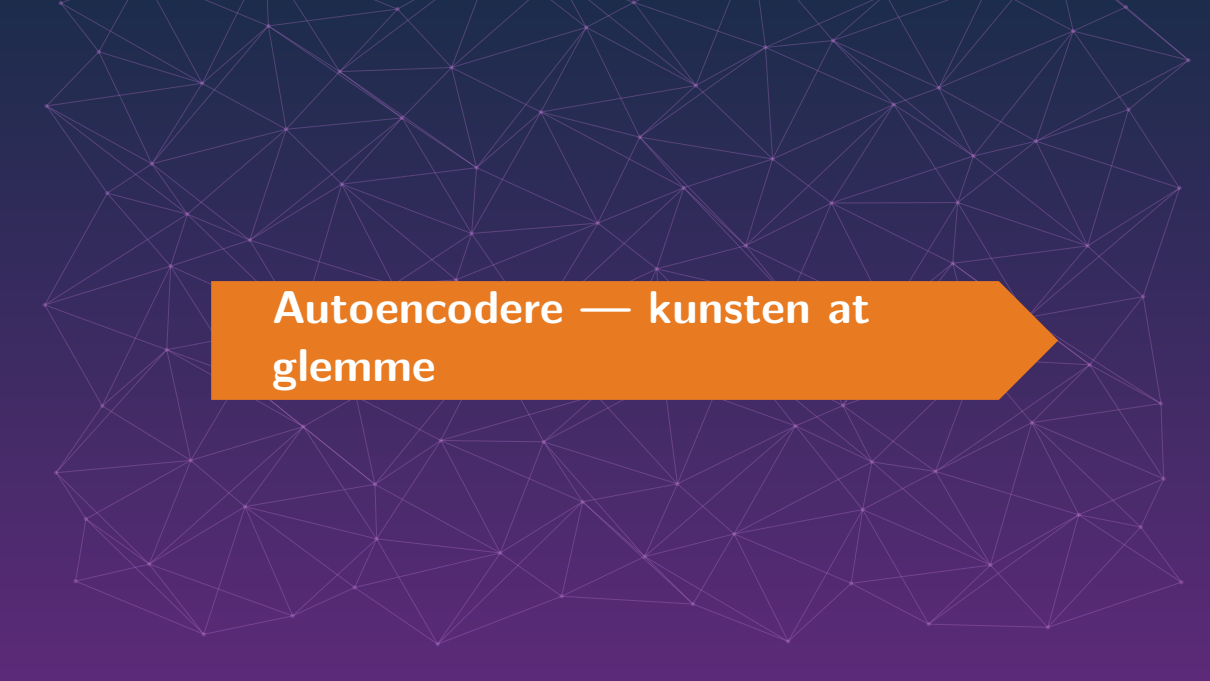
Fem værktøjer — vælg ét og dyk ned

UNF | KIC 2026



- I har nu grundmodellen: neurale netværk i PyTorch.
- I dag møder I fem **specialiserede** modeller — hver god til sin slags problem.
- Jeg giver en kort smagsprøve på hver (ca. 5 min).
- **Vælg så ét emne** og brug tiden på *opgaverne* — det er der, det sidder.

I skal ikke nå alle fem. I skal blive rigtig gode til ét.



Autoencodere — kunsten at glemme

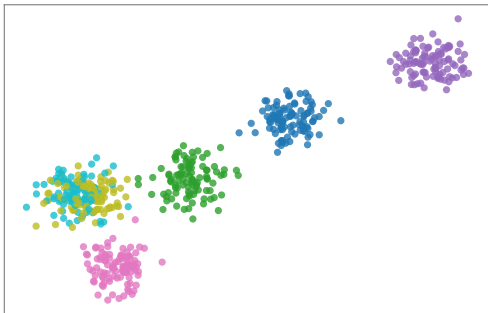


Autoencodere: pres data gennem en flaskehals

- Netværket skal genskabe sit **eget input** — men gennem en smal midte.
- Så kan det ikke snyde: det *må* lære, hvad der er vigtigt.
- **Ingen labels** — billedet er sit eget facit.

Tre superkræfter: kompression, støjfjernelse, anomali-detektion (“det her har jeg aldrig set før!”).

Latent rum: ens ting samler sig selv



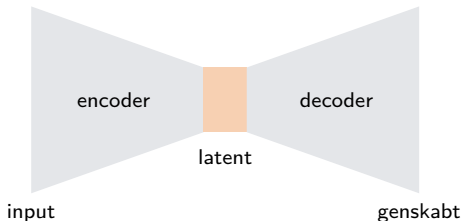
Den lærer selv at samle ens ting.

God til: kompression, alarm på det unormale — og forspil til transformers.



Sådan virker den: klem sammen, pak ud igen

- **Encoder** presser input ned i den smalle midte (få tal).
- **Decoder** pakker det ud igen til noget, der ligner originalen.
- Vi straffer forskellen — så midten *må* indeholde det vigtigste.
- Kan den ikke genskabe noget? Så er det **unormalt**.



Som at referere en hel film i én sætning — og så genfortælle filmen ud fra sætningen. Kan du ikke, har du ikke forstået den.



Beslutningstræer & Random Forests



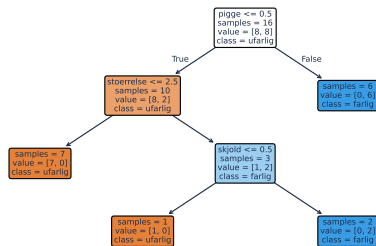
Beslutningstræer: 20 spørgsmål, lært fra data

- Ja/nej-spørgsmål, der deler dataen i renere bunker.
- Man kan faktisk **læse** det — modsat et neuralt netværk.
- **Random forest:** 100 træer stemmer → mindre overfitting.

På **tabeldata** er træ-modeller næsten altid det første, man prøver.

Netværkenes hjemmebane er billeder, lyd og sprog. Til pæne tabeller: brug et træ. Det rigtige værktøj til det rigtige job.

Et træ man kan **LÆSE** --- ét ja/nej-spørgsmål pr. lag



Er svampen giftig? Følg grenene.



Et lag = ét ja/nej-spørgsmål

Træet er bare en stak if'er — ét spørgsmål pr. lag:

```
if stoerrelse > 3:           # lag 1
    if pigge == 1:          # lag 2
        gaet = "FARLIG"     # lag 3
    else:
        gaet = "ufarlig"
else:
    gaet = "ufarlig"
```

- Hvert **lag** deler dataen i to renere bunker.
- Dybere træ = flere spørgsmål = finere opdeling.
- For dybt? Så lærer det udenad (overfit) — derfor en *forest*.

Ingen sorte kasser: du kan læse hvert eneste spørgsmål.

Det er bare “20 spørgsmål” — computeren har bare selv fundet ud af, hvilke spørgsmål der er værd at stille.



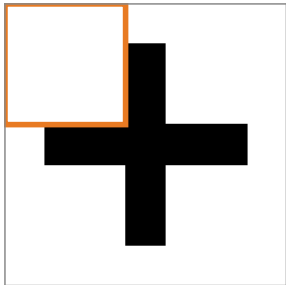
CNN — netværk der kan SE



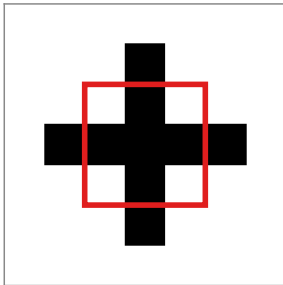
CNN: en lille kerne der glider hen over billedet

Samme lille kerne glider hen over hele billedet

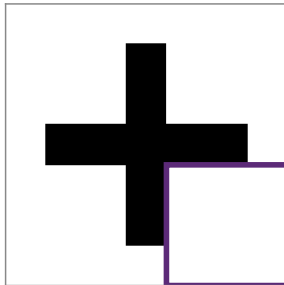
skridt 1



skridt 2



skridt 3



- En lille **kerne** (fx 3×3) glider hen over hele billedet.
- **Samme** kerne alle steder $\rightarrow$ få tal at lære, og den genkender mønstret uanset hvor det sidder.

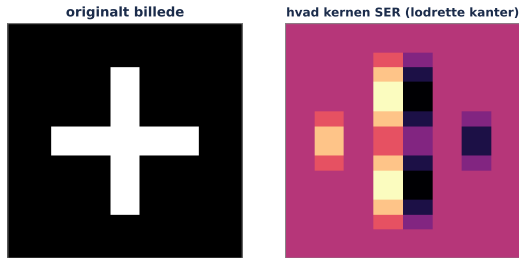
God til: billeder, video — alt hvor struktur betyder noget.



Hvad kernen fremhæver — og hvordan den bygger op

- Hver kerne leder efter ét mønster — fx en **lodret kant**.
- Stab mange lag: kanter → former → hele objekter.
- Præcis samme feature-idé som i går — bare med naboer i billedet.

CNN'er er grunden til, at computere i dag kan se.



Kernen fremhæver lodrette kanter.

Én kerne finder kanter. Den næste finder øjne. Den næste finder katte.
Ingen har programmeret “kat” — det faldt bare ud af det.



Clustering — mønstre UDEN labels

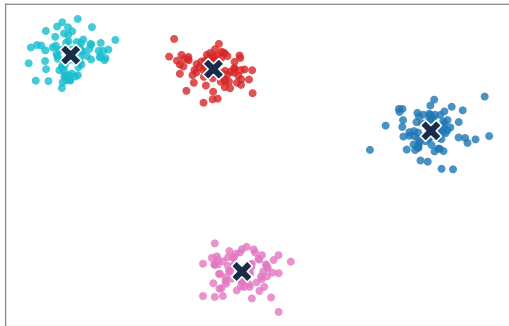


Clustering: find grupper uden facitliste

- Ingen labels — bare rå observationer. Det hedder **unsupervised learning**.
- **k-means**: placér k centre, træk hvert punkt til nærmeste, flyt centrene, gentag.
- Ingen gradienter, ingen tabsfunktion — bare afstande og gennemsnit.

“Hvilke kundetyper har vi egentlig?” — uden på forhånd at vide det.

Clustering: grupper UDEN facitliste



Krydsene er de fundne centre.

God til: kunde-segmenter, opdage grupper man ikke vidste fandtes.



Sådan finder k-means grupperne

- 1 Vælg k (antal grupper) og kast k centre tilfældigt ud.
- 2 Tildel hvert punkt til det **nærmeste** center.
- 3 Flyt hvert center hen i **gennemsnittet** af sine punkter.
- 4 Gentag trin 2–3, indtil intet flytter sig mere.

Ingen gradienter, ingen labels — bare afstande og gennemsnit.

Du skal selv gætte k . Vælger du $k = 2$ til tre grupper, maser den bare to af dem sammen — og ser fjollet ud. Man prøver sig frem.



RNN & LSTM — modeller med hukommelse

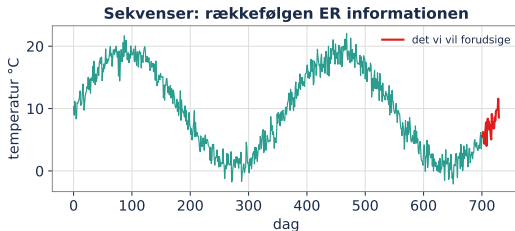


Sekvenser: når rækkefølgen ER informationen

- Tidsserier, melodier, sensordata — og frem for alt **sprog**.
- En **RNN** læser skridt for skridt og bærer en lille *hukommelse* med.
- **LSTM** er den forbedrede version: den lærer at huske og glemme.

“Gæt næste ord” er præcis “gæt morgendagens temperatur”.

Dette er broen til i morgen: sprog er bare en sekvens — og det er dét, transformers æder til morgenmad.

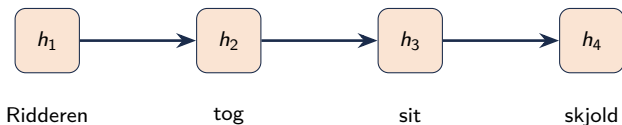


11 års temperatur — se årstiderne.



Sådan husker en RNN: ét skridt ad gangen

hukommelsen bæres videre →



- Den læser ét ord ad gangen og opdaterer en lille **hukommelse** (h).
- Hukommelsen bærer konteksten med videre til næste ord.
- **LSTM** er en RNN, der er blevet god til at huske det vigtige og glemme resten.

Problemet: efter en lang sætning har den glemt starten — ligesom mig midt i en lang tavle-udregning. Det er præcis dét, transformers løser i morgen.



Model	Bedst til
Autoencoder	kompression, støjfjernelse, anomali-alarm
Beslutningstræ / forest	pæne tabeller (og man kan læse svaret)
CNN	billeder og alt med rumlig struktur
Clustering (k-means)	grupper uden labels
RNN / LSTM	sekvenser: tid, lyd, sprog

Der findes ikke én model til alt — der findes det rigtige valg.



Åbn mappen 04-Specialiserede-Modeller/

Vælg det emne, der fænger — og brug tiden på opgaverne.

- | | |
|------------------------|---------------------------------------|
| 1-Autoencoders.ipynb | se det latente rum sortere cifre selv |
| 2-Decision-trees.ipynb | tør du spise svampen? |
| 3-CNN.ipynb | byg et net der ser tøj |
| 4-Clustering.ipynb | find kundetyper i et indkøbscenter |
| 5-RNN-sequences.ipynb | slå "i morgen = i dag"-vejrudsigten |

The background is a gradient from dark blue at the top to a deep purple at the bottom. Overlaid on this is a complex, white line network that forms a series of interconnected triangles and polygons, resembling a wireframe or a molecular structure. The lines are thin and create a sense of depth and connectivity.

Vælg ét værktøj — og bliv rigtig god til det.